

Trabalho Prático de Programação Modular – Parte 1

Todo o código do programa deve estar organizado em um pacote cujo nome fica a escolha do grupo. Dentro desse pacote deve ficar uma classe chamada Programa, que possui o método main e inicializa o programa. Dentro do pacote também deve haver os pacotes citados a seguir. O código pode ser implementado em qualquer linguagem orientada a objetos.

OBS.: neste documento é descrito como exemplo um programa de venda de produtos. Vocês não podem implementar esse programa. Pensem em outra aplicação.

Pacote modelo

Deve ser criada uma classe abstrata chamada **Entidade**, que possui um atributo chamado id (inteiro), que serve como identificador.

- construtor vazio e construtor que recebe o id;
- getters e setters (em Python não é obrigatório);
- toString (__str__ em Python);

Neste pacote devem ser criadas, no mínimo, 3 subclasses de Entidade (por exemplo: Cliente, Produto, etc.). Uma delas deve ser uma transação (por exemplo: Venda, Locação, Manutenção, Consulta, etc.). Cada subclasse de Entidade deve possuir um relacionamento de associação com, pelo menos, uma outra (por exemplo, venda possui um atributo do tipo cliente). Também é obrigatório usar o comando super nos métodos sobrescritos.

Na classe da entidade de transação, deve haver uma composição 1 para muitos com outra entidade, gerando uma outra classe intermediária (uma quarta classe) com, pelo menos, um atributo próprio. Por exemplo, venda pode ter um ou mais itens. Portanto, uma classe ItemVenda, possui como atributos o Produto e a quantidade que foi vendida deste. A classe da transação deve possuir métodos para adicionar e remover objetos dessa classe intermediária. Os objetos da classe intermediária são salvos junto com o objeto da classe de transação que os contém.

Pacote dados

Deve ser criada a classe **EntidadeDAO**, que possui um atributo que é um conjunto (Set) do tipo Entidade que armazena todos os objetos salvos da entidade em questão e os seguintes métodos:

- **salvar**, que recebe um objeto do tipo Entidade e o adiciona no conjunto. Não pode haver duas entidades do mesmo tipo com o mesmo id. Se isso ocorrer (ou qualquer outro problema), o objeto não é adicionado no conjunto e o método retorna false. Se tudo der certo, o método retorna true;
- **atualizar**, que recebe um objeto do tipo Entidade que substitui o elemento existente no conjunto que possui o mesmo id. Retorna false, se não existir um objeto com mesmo id no conjunto, ou true, se existir;
- **apagar**, que recebe um id e remove e retorna o elemento existente no conjunto que possui o mesmo id. Retorna null se não existir um objeto com o id indicado no conjunto;
- **buscar**, que recebe um id retorna o objeto no conjunto que possui o id indicado. Retorna null se não existir um objeto com o id indicado no conjunto;
- **carregar**, que retorna um array (array mesmo, não lista nem o próprio conjunto) com todos os objetos armazenados no conjunto ordenados por id;
- **persistir**, que persiste o conteúdo do conjunto em um arquivo.
- **recuperar**, que carrega o conjunto com as entidades contidas em um arquivo.

Crie uma classe que garante que exista apenas uma única EntidadeDAO por entidade em todo o sistema e, dessa forma, impedindo ambiguidades na persistência de cada entidade. As classes dos demais pacotes realizam as operações de persistência a partir dessa classe.

Pacote visao

O programa deve apresentar uma interface em linha de comando. Ao executar o programa a interface deve apresentar um menu com uma opção para sair do programa e uma opção para cada entidade (a classe intermediária da composição não conta aqui).

Ao selecionar uma das opções que representa uma entidade, deve aparecer um outro menu com uma opção para voltar ao menu inicial e opções para inserir, alterar, apagar, visualizar a partir do id e visualizar todos. Os objetos devem ser armazenados na classe que representa o banco de dados.

No caso da entidade que é a transação, as opções de inserir e alterar devem permitir adicionar ou remover objetos da classe intermediária. Por exemplo, em uma venda, deve ser possível adicionar e remover produtos, solicitando do usuário o id do produto e a quantidade comprada. Isso deve gerar objetos ItemVenda, que são adicionados/removidos da Venda em questão.